Large Language Models Compression: Distillation, Quantization and Pruning

Agenda

- Introduction to model compression
- Notebook 0: Baseline for model compression
- Notebook 1: Quantization
- Notebook 2: Pruning
- Notebook 3: Knowledge distillation
- Notebook 4: Model deployment and benchmarking

Introduction to Model Compression

LLM Inference

Why is it Challenging?

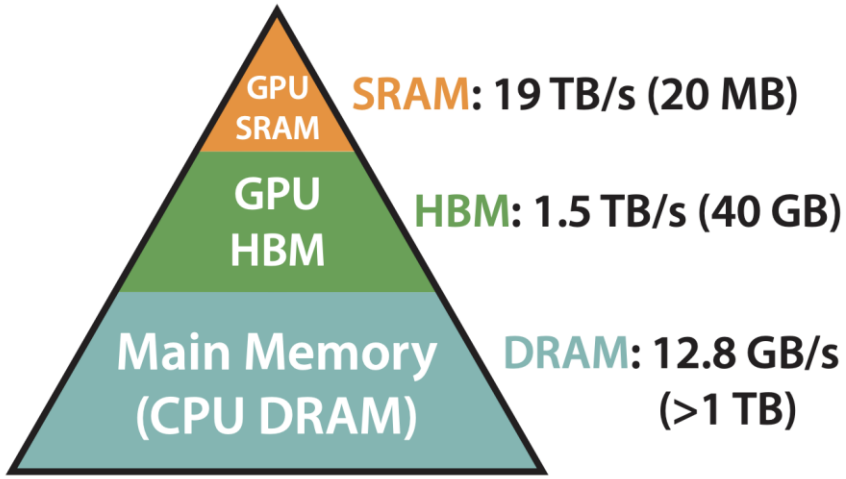
Memory

Compute

Memory for Inference

Even small LLMs are large

- Each billion parameters is ~2GB of memory
- Llama 8B is ~16GB of memory + the KV cache
- A H100 has 80GB of memory and finite bandwidth
- How can we make the most out of this memory?



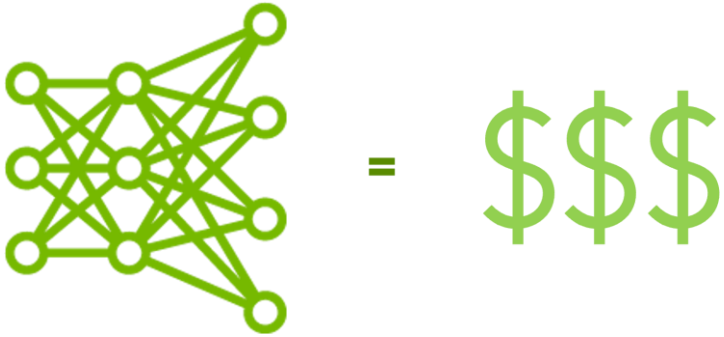
Memory Hierarchy with Bandwidth & Memory Size

Image from book "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness".2022

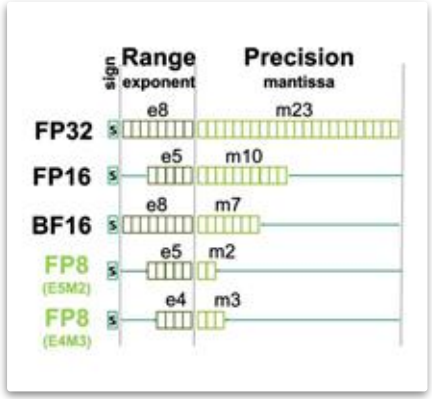
Operations for Inference

Billions of Operations Increase the Cost

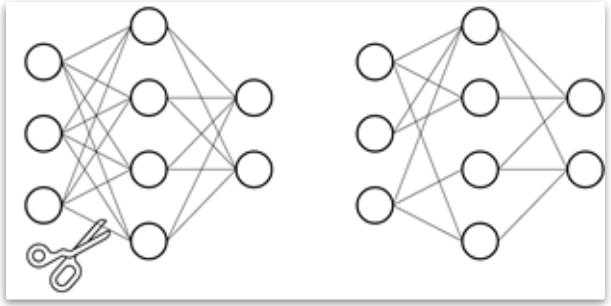
- Larger models perform better, but are costly
- Smaller LLMs can be a good trade-off between cost and quality
- More efficient models drive the cost of inference down
- Can we make the inference computations cheaper?



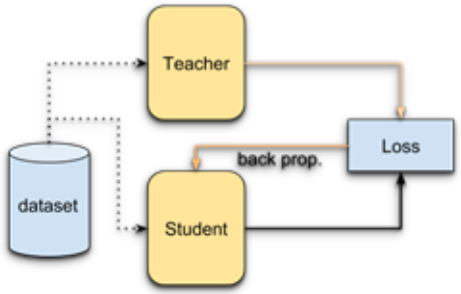
Model Compression Strategies



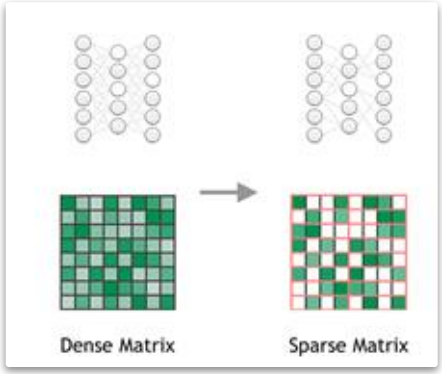
1. Quantization



2. Pruning



3. Distillation



4. Sparsity

Distilling the Knowledge of LLMs into SLMs

Train only the largest LLM and get smaller models with similar quality

How to Prune and Distill Llama-3.1 8B to an NVIDIA Llama-3.1-Minitron 4B Model

Aug 14, 2024

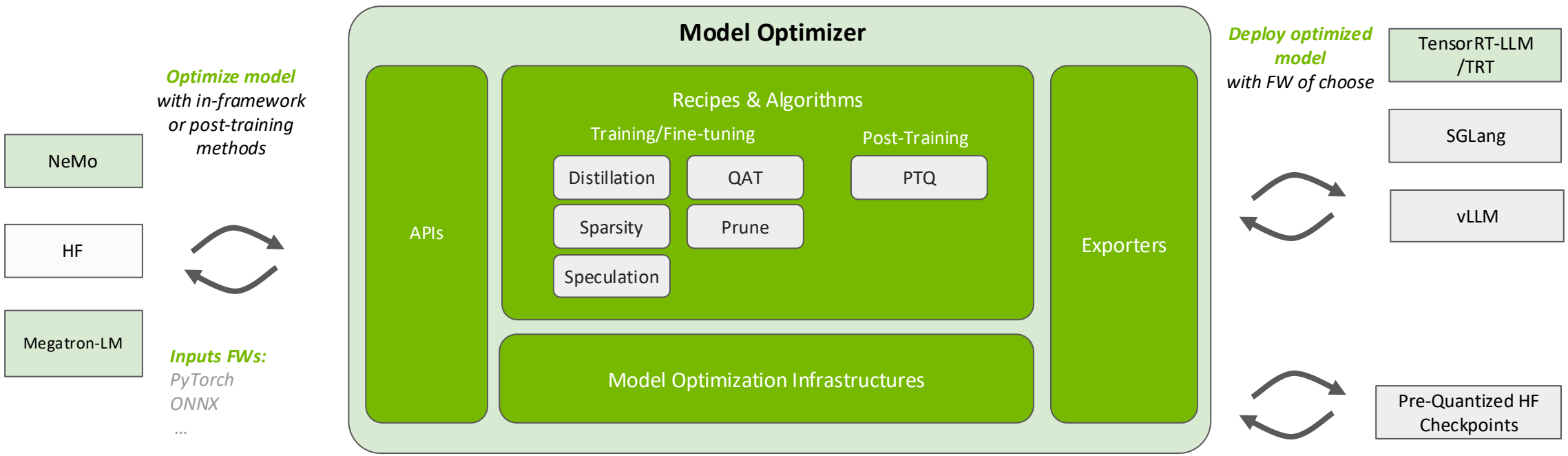
+32 Like Discuss (5)

By Sharath Sreenivas, Vinh Nguyen, Saurav Muralidharan, Marcin Chochowski and Raviraj Joshi

Blog https://developer.nvidia.com/blog/how-to-prune-and-distill-llama-3-1-8b-to-an-nvidia-llama-3-1-minitron-4b-model/
Paper https://arxiv.org/pdf/2407.14679

NVIDIA TensorRT Model Optimizer

Product Overview

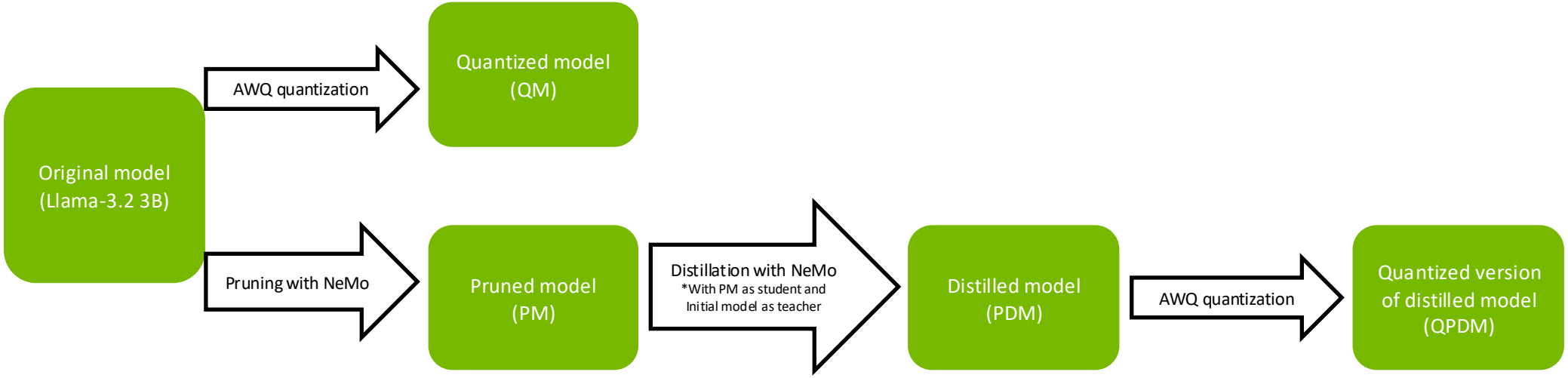


Nvidia AI SW Stack

TensorRT-Model-Optimizer
HF Checkpoints

Model compression in this course

Path to different model version by applying the techniques introduced



Quantization

TFLOPS

50x

GPU	FP32
Fermi (2010)	1.6
Pascal (2016)	10.6
Volta (2017)	14.9
Ampere (2020)	19.5
Hopper (2022)	66.9
Blackwell (2024)	80.0

GPUs getting faster for AI

TFLOPS

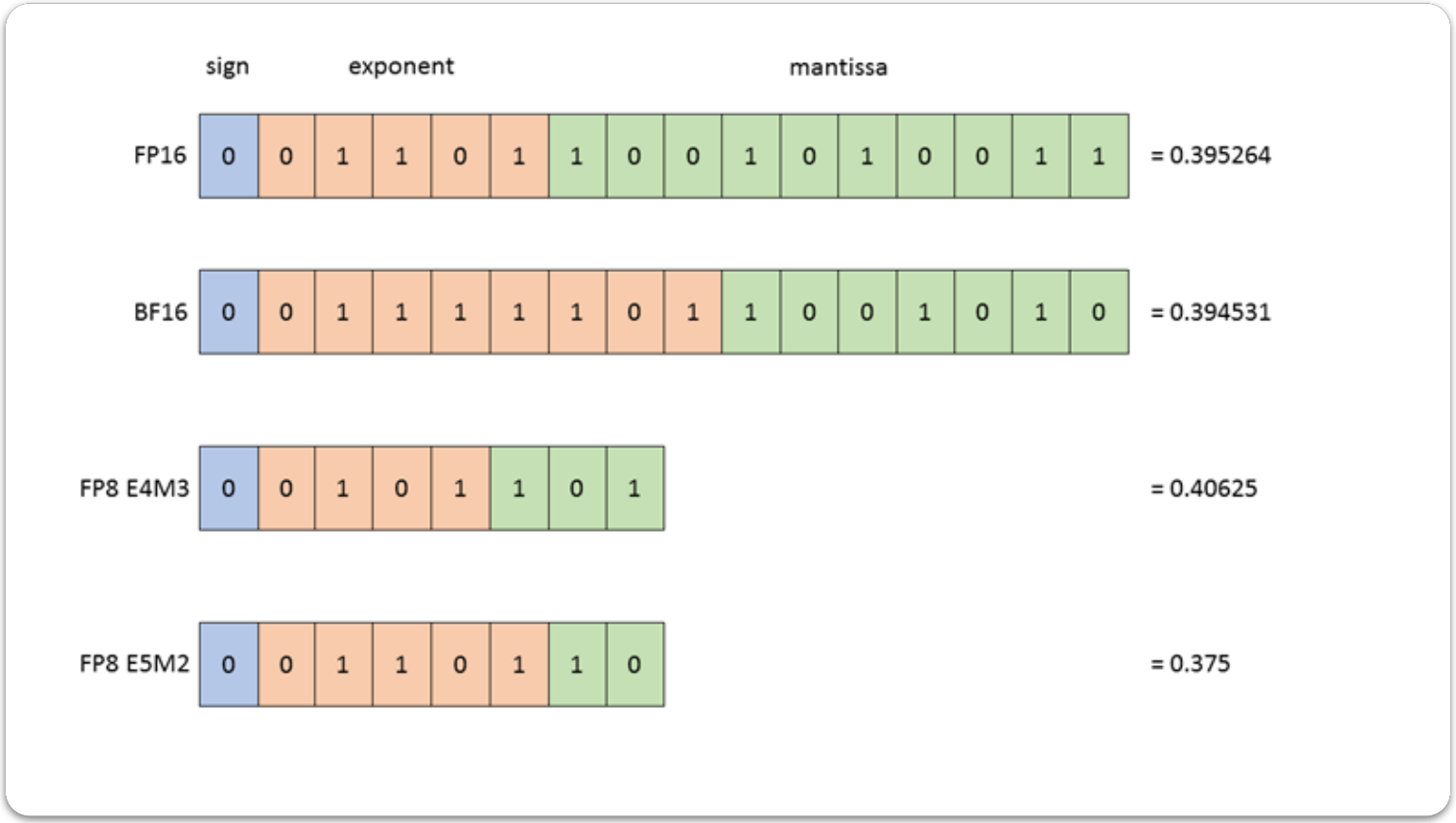
GPU	FP32	TF32	FP16	BF16	FP8	INT8	FP6	FP4
Fermi (2010)	1.6							
Pascal (2016)	10.6		21					
Volta (2017)	14.9		119					
Ampere (2020)	19.5	156	312	312		624		
Hopper (2022)	66.9	535	989	989	1,979	1,979		
Blackwell (2024)	80.0	1,100	2,250	2,250	4,500	4,500	4,500	9,000

50x

112.5x

What does it mean to use less bits?

This is called quantization



Benefits of less bits

Memory

Weights and tensors occupy less space in memory

Bandwidth

Faster data movement from main memory HBM to cores (and vice versa)

Compute

More TFLOps with less bits

Faster matrix multiplications

Quantization

How to Choose a Precision?

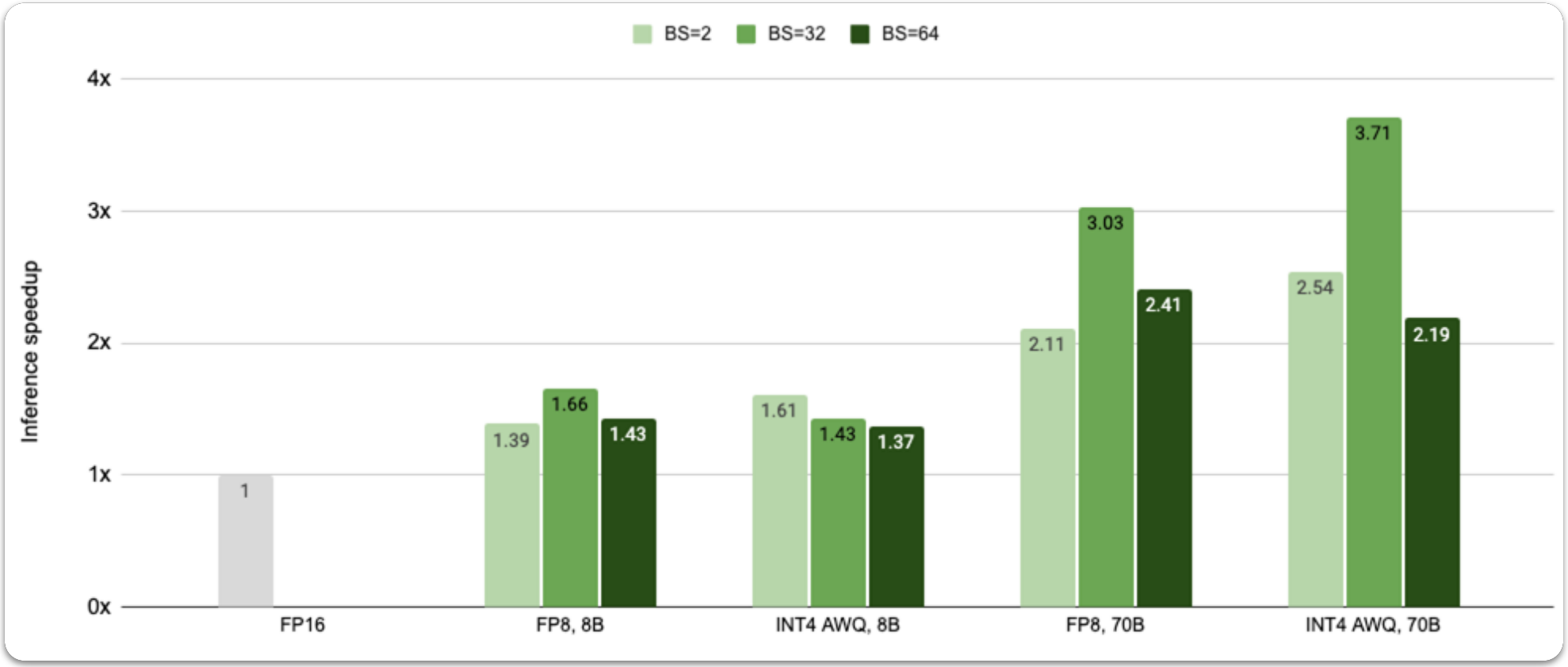
- Best precision varies by application
  - FP format is better than INT for accuracy of LLMs
- Weight quantization reduces memory footprint & traffic
  - Reduces latency
  - Can fit larger models
  - Costs compute time to unpack the weights
- Activation quantization saves on compute
  - Improves throughput
  - Can run larger batch sizes
- W*X*A*Y* = weights quantized to *X* bits, and activations to *Y*

Method	Performance Improvement		Accuracy impact	Calibration time
	small batch BS <=4	large batch BS>=16		
FP8 (W8A8)	Medium	Medium	**Very low / None**	**O(1min)**
INT8 SQ (W8A8)	Medium	Medium	Medium	O(1min)
INT8 WO (W8A16)	Medium	***None***	**Low**	***None***
INT4 WO (W4A16)	**High**	***None***	**High**	***None***
INT4 AWQ (W4A16)	**High**	***None***	**Low**	O(10min)
INT4 GPTQ (W4A16)	**High**	***None***	**Low**	O(10min)
INT4-FP8 AWQ (W4A8)	**High**	Medium	**Low**	O(10min)

SQ = Smooth Quant
WO = Weight Only
AWQ = Activation Aware Quantization

Quantization Guide

Inference speedup with Model Optimizer PTQ for Llama3 7B and 70B

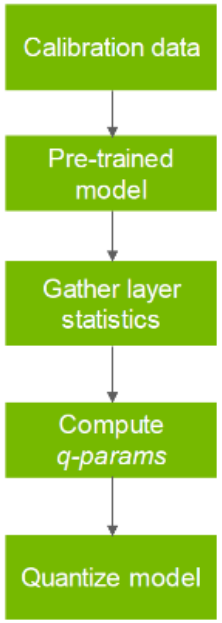


Full Benchmark Results

Recovering Quality After Quantization

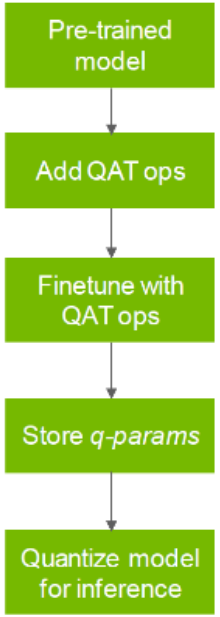
Approaches

Post-training quantization(PTQ)



Post-Training Quantization (PTQ)		Quantization-Aware Training (QAT)
Start with a pre-trained model and evaluate it on a calibration dataset.		Start with a pre-trained model and introduce quantization ops at various layers
Calibration data is used to calibrate the model. It can be a subset of training data.		Finetune it for a small number of epochs.
Calculate dynamic ranges of weights and activations in the network to compute quantization parameters (q-params).		Simulates the quantization process that occurs during inference.
Quantize the network using q-params and run inference		The goal is to learn the q-params which can help to reduce the accuracy drop between the quantized model and pre-trained model.

Quantization-aware training (QAT)



FP8 Quantization

Post-training quantization is enough

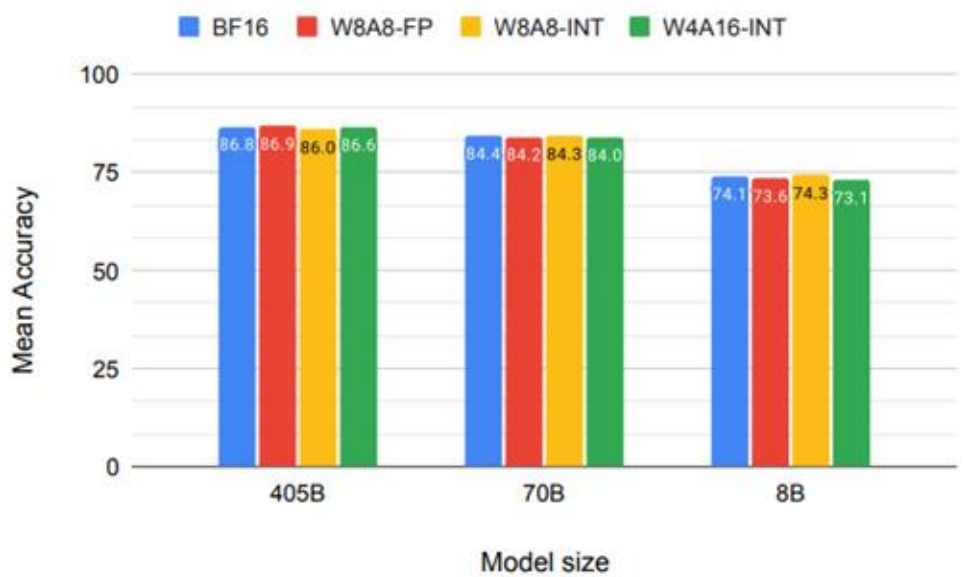
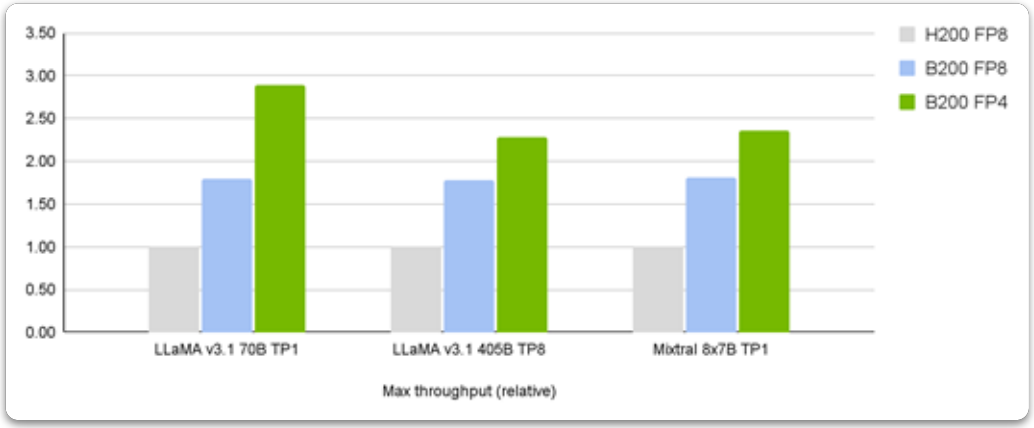


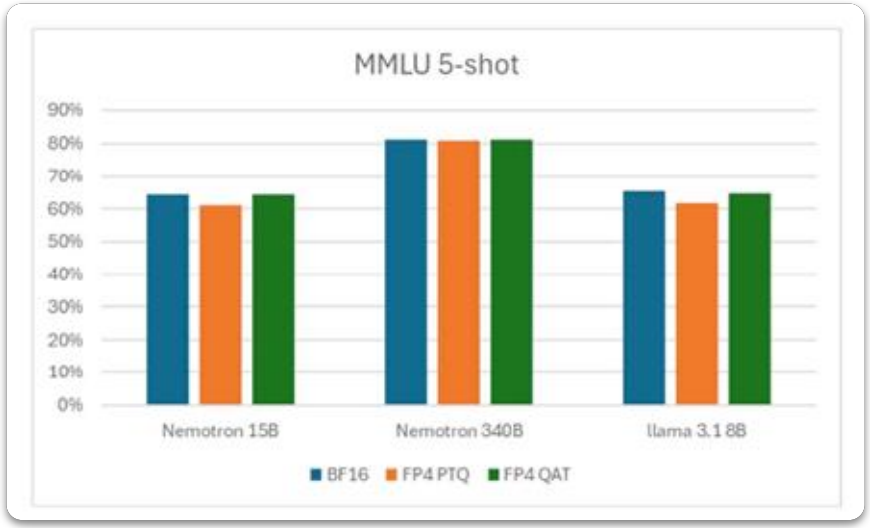
Figure 1. Average score of quantized Llama-3.1-Instruct models on the Open LLM Leaderboard V1 benchmark.

Chart From "Give me BF16 or give me death"? Accuracy-Performance trade-offs in LLM quantization

FP4 for Blackwell



Speed



Quality

More details in GTC session [Enable Blackwell Inference With TensorRT Model Optimizer]

The Lab

Notebooks 0 + 1

- **Notebook 0:**
  - Setting up the baseline for the lab – Llama-3.2-3B in BF16

NVIDIA

Notebook 0: Introduction to Model Compression

Motivation

The recent advances in generative AI have seen tremendous progress in the capabilities of Large Language Models (LLMs) on theoretical benchmarks and industry applications. With LLM sizes ranging from a few billion to over one trillion parameters, practitioners face significant challenges with efficient deployment, inference-time latencies and operational costs.

- **Notebook 1:**
  - Quantization of nemo models into FP8
  - Building a TRT-LLM engine from the quantized checkpoint
  - Evaluating the engine in MMLU in 4 sub-categories

NVIDIA

Notebook 1: Quantization

In this notebook you will learn about:

1. Building optimal inference engines for the `Llama-3.2-3B` model using TensorRT-LLM
2. Using the quantization toolkit in TensorRT-LLM to quantize your model into different precisions
3. Evaluating the performance of your quantized engines using the MMLU benchmark

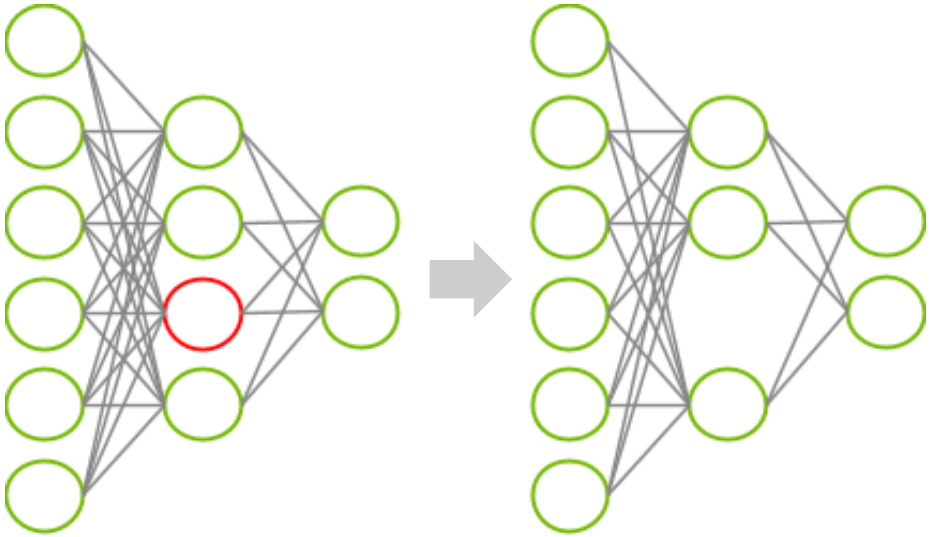
Pruning

Model Optimization

Pruning

- Reduce the complexity of neural networks by removing unnecessary nodes and connections
  - Reduce memory bandwidth
  - Reduce memory footprint
  - Accelerate the compute

- **Challenge**:
  - Remove nodes and edges from a network in such a way as to minimize the accuracy loss of the smaller resulting model compared to the original unpruned model.

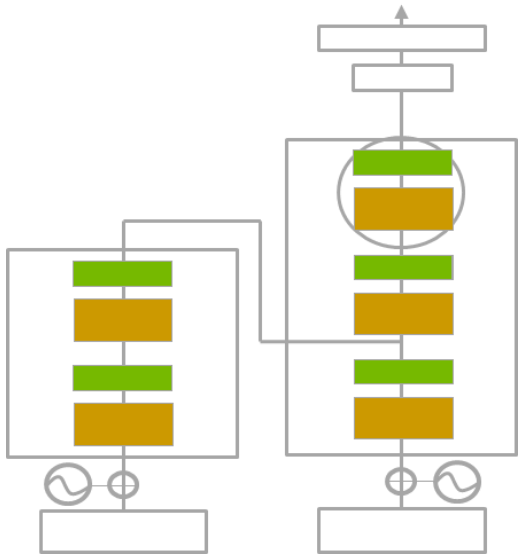


6 inputs, 6 neurons (including 2 outputs), 32 connections

6 inputs, 5 neurons (including 2 outputs), 24 connections

Importance Analysis

For a Transformer Architecture



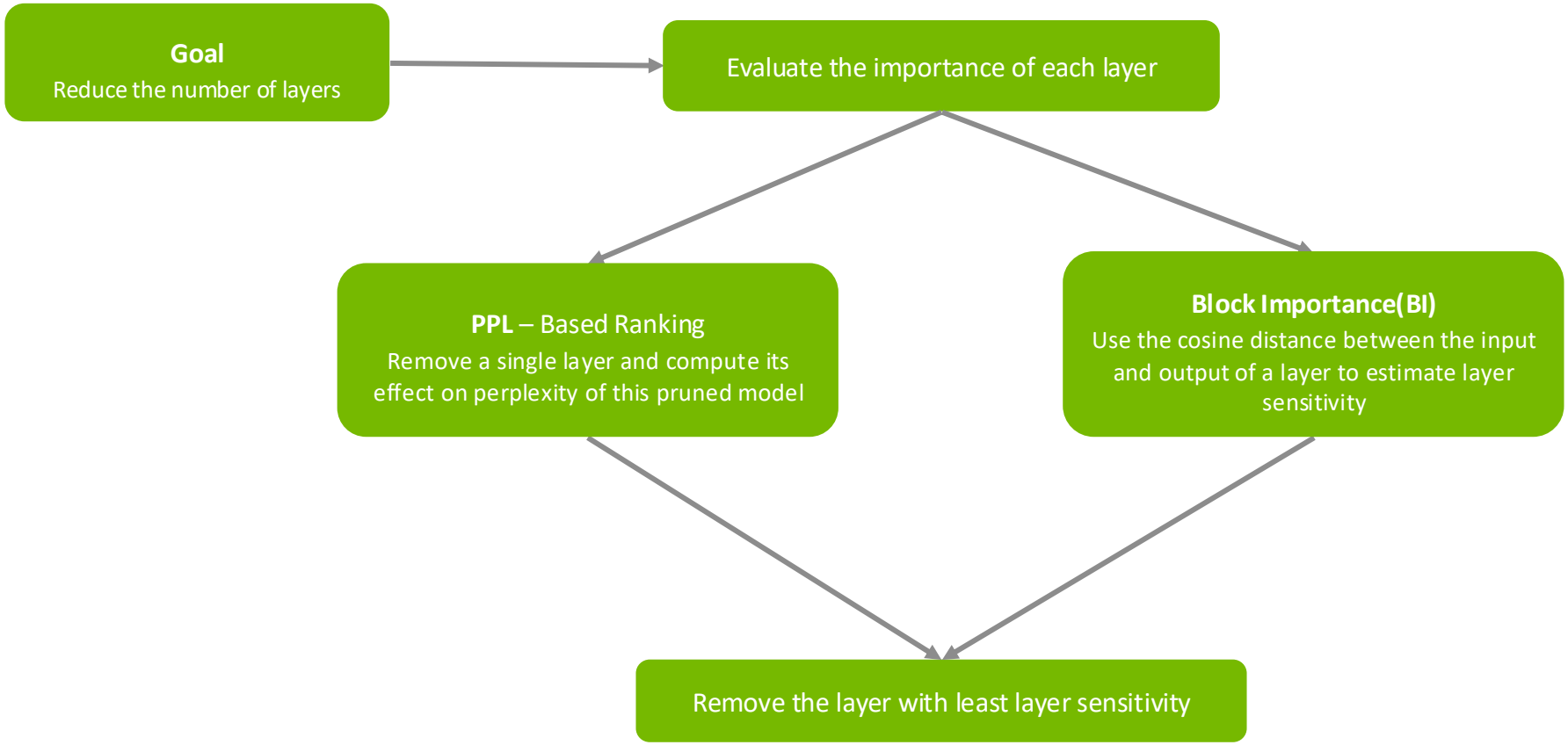
- Using gradients is impractical in a model with billion parameters
- Instead, use activation-based importance estimation strategy
- Simultaneously compute sensitivity information for all the axes:
  - Layers
  - Neurons
  - Attention heads
  - Embedding channels
- For **depth pruning** aim to drop a continuous subgroup of layers that results in the least accuracy drop
- For **width pruning** aim to meet the chosen parameter count* by dropping neurons, attention heads, and embedding channels based on importance ranking

*The parameter count is often dictated by practical latency requirements or amount of available hardware

More details in LLM Pruning and Distillation in Practice: The Minitron Approach

Depth Pruning

Reduce number of **Layers** in a Neural Network



Width Pruning

Reduce the number of attention heads, neurons and embedding channels in a neural network to make it smaller

Neural Architecture Search

Goal: Reduce the number of **attention heads, neurons and embedding channels**

Analyse which neurons or filters are the least important

Compute activation-based importance scores for attention heads, neurons, and embedding channels

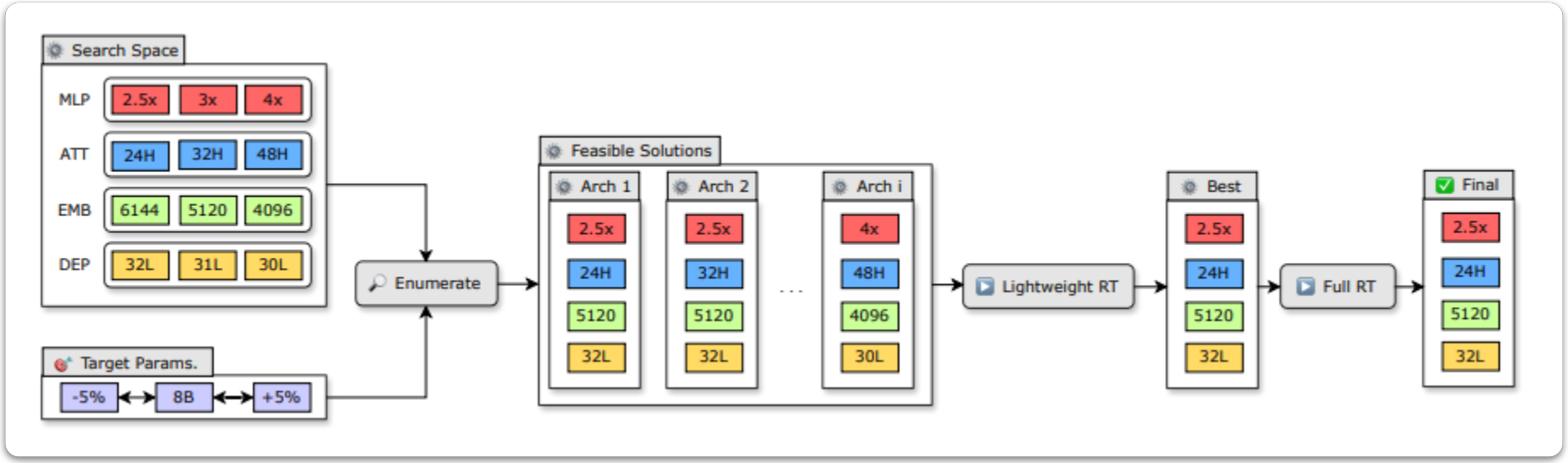
$$F_{\text{head}}^{(i)} = \sum_{\mathbf{B},\mathbf{S}} \| \text{Attn}(\mathbf{X}\boldsymbol{W}^{Q,i}, \mathbf{X}\boldsymbol{W}^{K,i}, \mathbf{X}\boldsymbol{W}^{V,i}) \|_2$$

$$F_{\text{neuron}}^{(i)} = \sum_{\mathbf{B},\mathbf{S}} \mathbf{X}(\boldsymbol{W}_1^i)^T, F_{\text{emb}}^{(i)} = \sum_{\mathbf{B},\mathbf{S}} LN(\mathbf{X})_i$$

Drop the elements with least activation scores

More details in [LLM Pruning and Distillation in Practice: The Minitron Approach]

Neural Architecture Search Algorithms



- Goal: arrive at a set of feasible architectures meeting the parameter budget
- In practice, this step can be skipped and approximated by simpler search methods

*RT refers to retraining

More details in [Puzzle: Distillation-Based NAS for Inference-Optimized LLMs]

The Lab

Notebook 2

Notebook 2: Pruning

In this notebook you will learn about:

- The theoretical intuition for the two main pruning methods: depth and width pruning
- How to create pruned models from an LLM using the NeMo framework
- Evaluate the pruned model using the MMLU benchmark

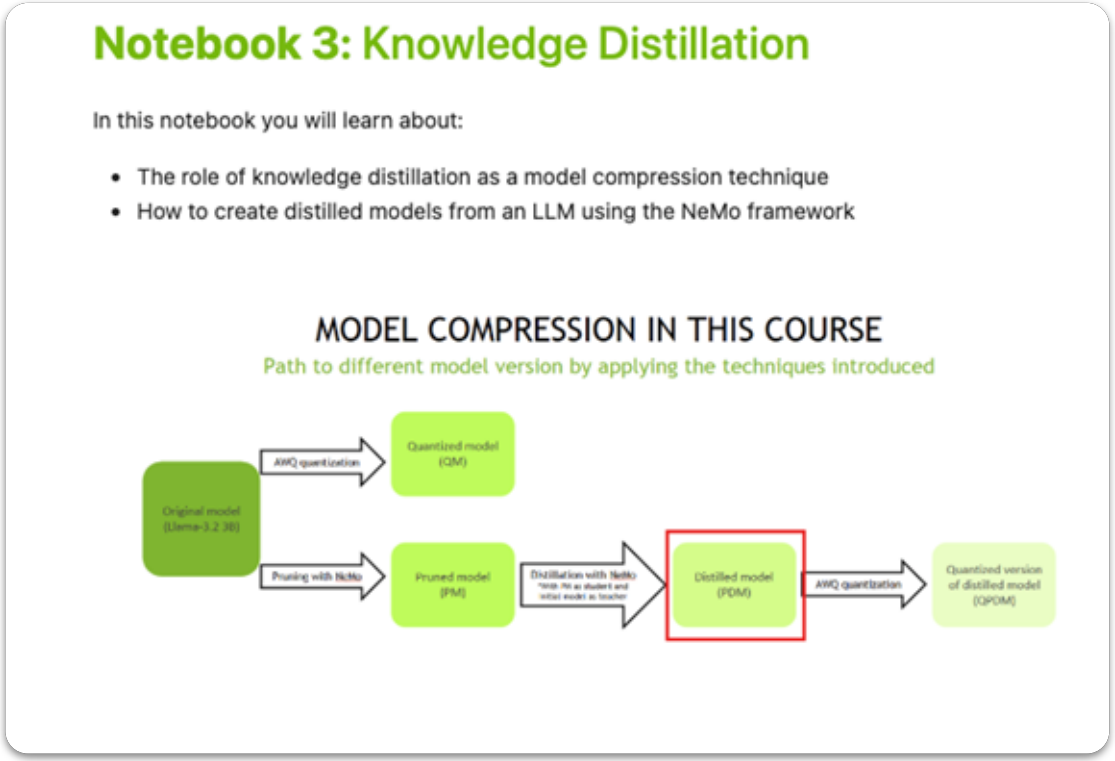
MODEL COMPRESSION IN THIS COURSE

Path to different model version by applying the techniques introduced

1. Apply depth-pruning to **Llama-3.2-3B** to prune 25% of the layers
2. Evaluate the pruned model on MMLU benchmark

The Lab

Notebook 3

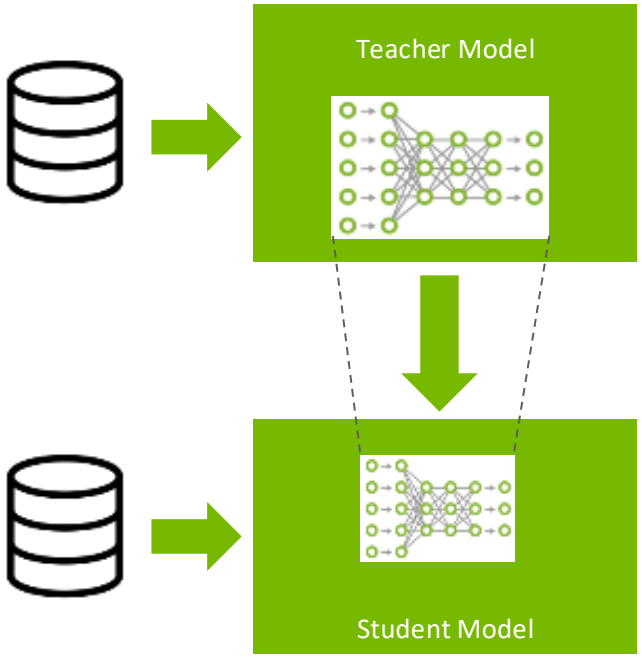


- Before continuing with the lecture, kick off the knowledge distillation in notebook 3
- The distillation takes around **20 mins**

Knowledge Distillation

Knowledge Distillation

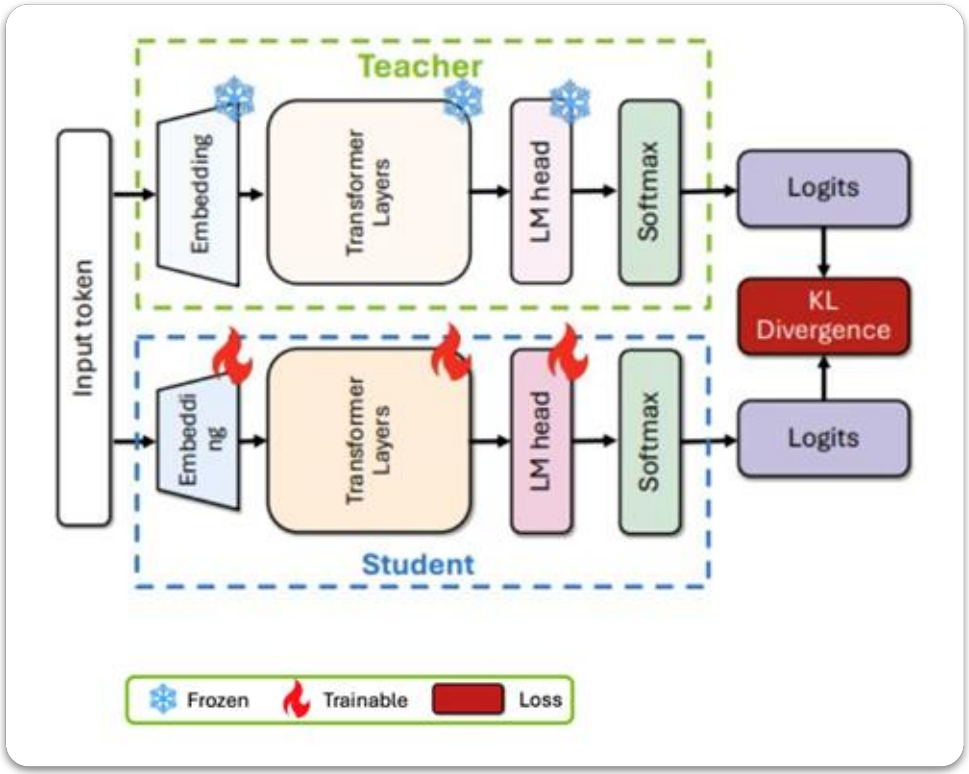
The Underlying Idea



- Knowledge is transferred from a larger Teacher model to a smaller Student model
  - Hard distillation – students learn from the outputs of the teacher
  - Soft distillation – student learn from the internal state of the teacher
- **Soft distillation**
  - Student receives more detailed supervision from the teacher's internal states (logits) than from the outputs only
  - Requires less training data for the supervision
  - Can seamless be incorporated as part of multi-step iterative model compression process

We focus exclusively on soft distillation in this course.

Soft Distillation

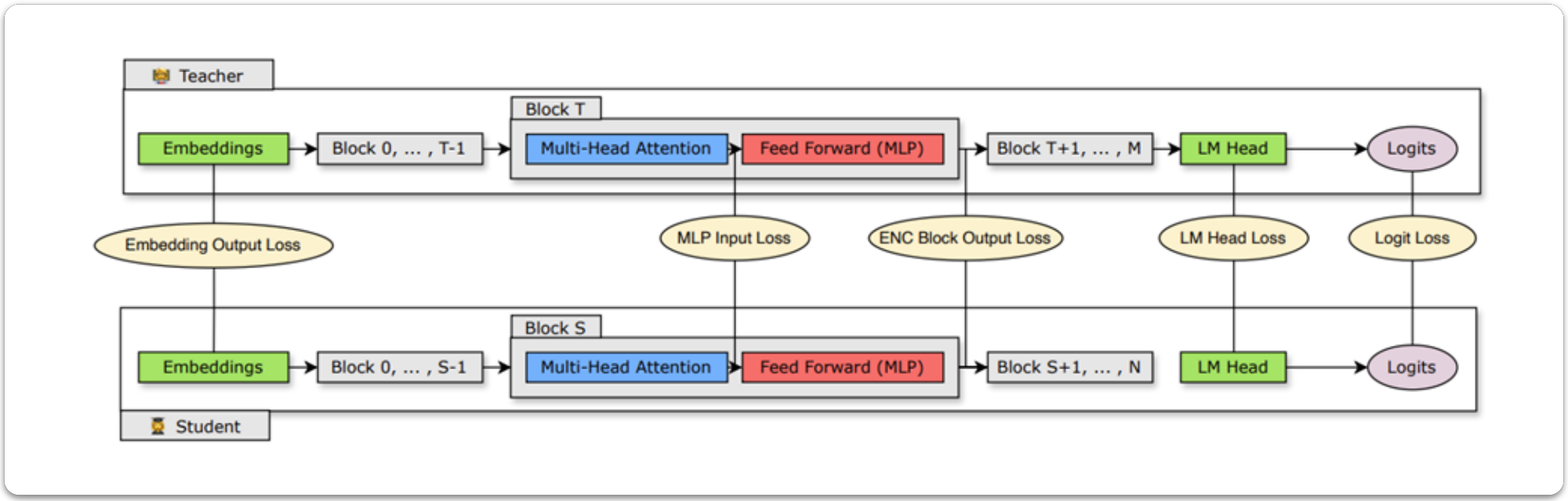


- The training is done on a chosen dataset
- Teacher weights remain frozen
- Student learns from the teacher's logits
    - Specifically, the student's weights get updated at each step to minimize the KL divergence between its logits and the logits of the teacher
- The process is repeated for one or a few epoch – passes over the dataset, until convergence

- **Definition**: KL divergence is a statistical distance measuring how much a probability distribution (here the student's) differs from a true probability distribution (here the teacher's).

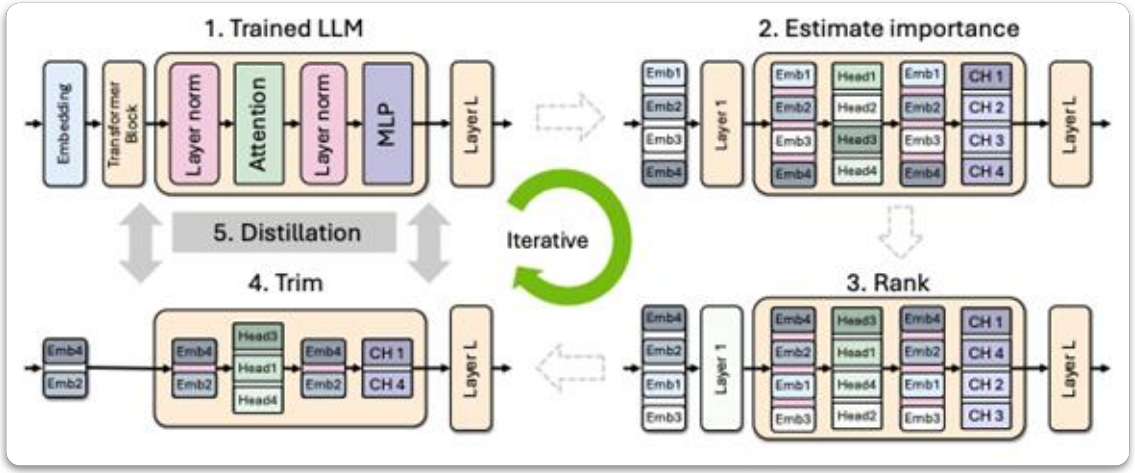
Knowledge Distillation - Transformer

A detailed view of knowledge distillation steps for a transformer network



The Minitron Approach

Iteratively Apply Pruning and Distillation



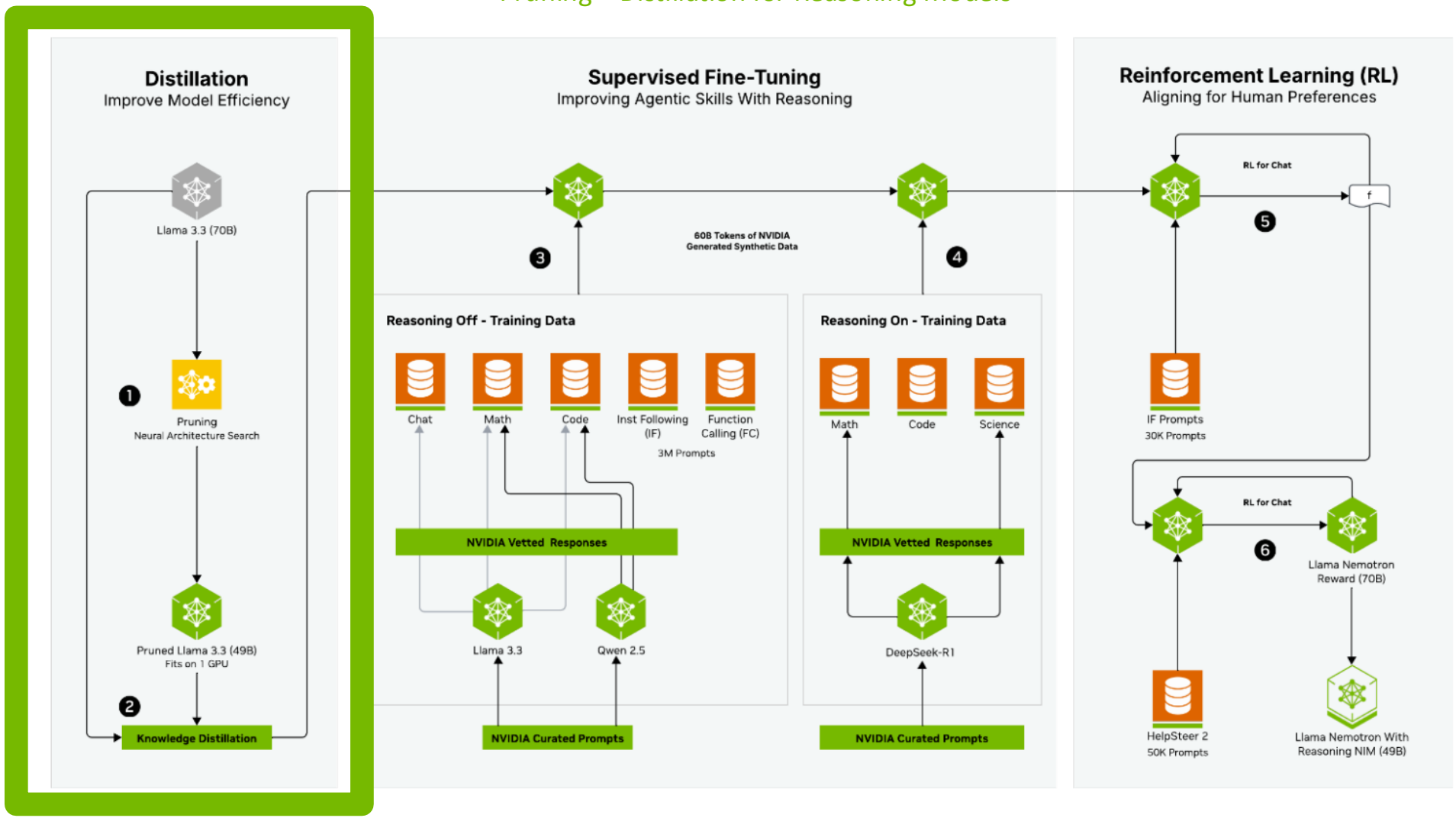
- Start with a pre-trained LLM

1. Evaluate importance of neurons, rank them
2. Pruning - trim the least important neurons
3. Distill the knowledge from the original LLM to the pruned model
4. The original model is replaced with the distilled model for the next iteration of compression

(NVIDIA) Compact Language Models via Pruning and Knowledge Distillation. Muralidharan et al. 2024 2407.14679

Llama Nemotron Models

Pruning + Distillation for Reasoning Models



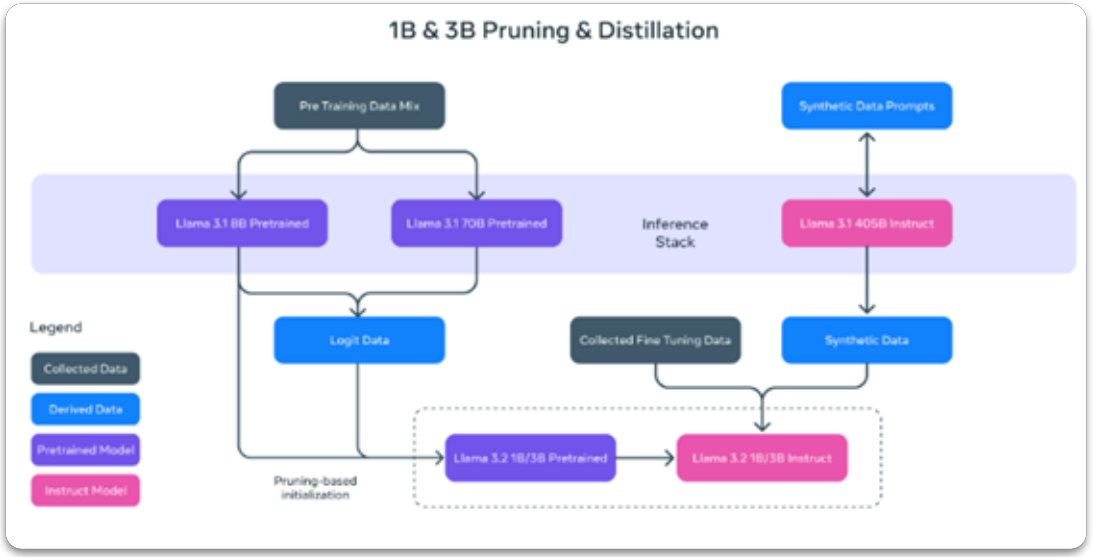
Examples of Distilled Model Families

DeepSeek-R1Models

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	**1820**
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	**72.6**	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	**86.7**	**94.5**	**65.2**	**57.5**	1633

Deepseek-R1 paper by DeepSeekAI

Llama-3.2 Models



Llama-3.2 blog by Meta

The Lab

Notebook 3

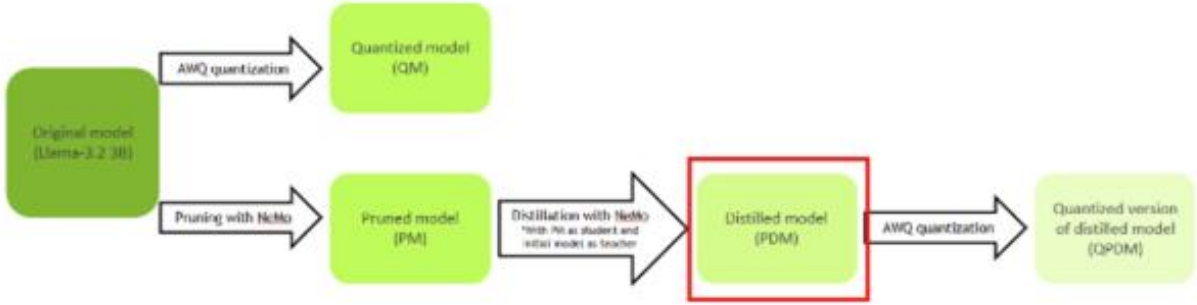
Notebook 3: Knowledge Distillation

In this notebook you will learn about:

- The role of knowledge distillation as a model compression technique
- How to create distilled models from an LLM using the NeMo framework

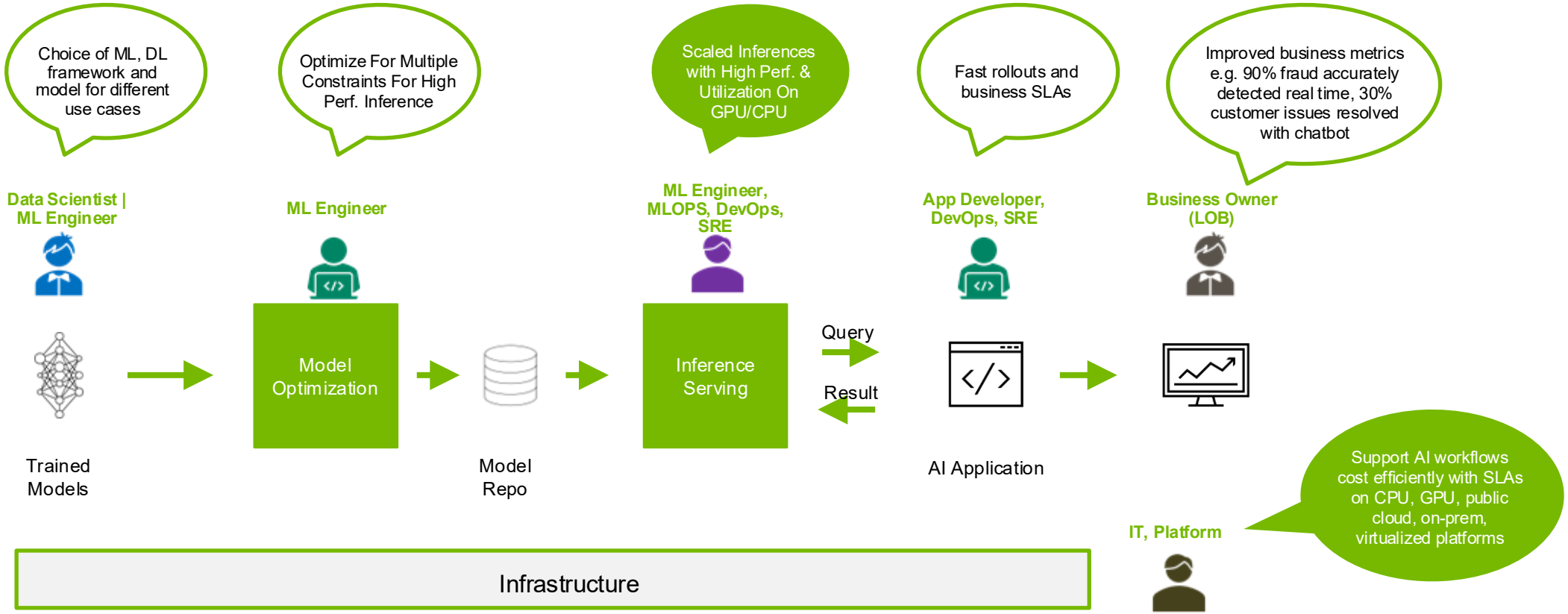
MODEL COMPRESSION IN THIS COURSE

Path to different model version by applying the techniques introduced



Model Deployment and Benchmarking

AI Inference Workflow

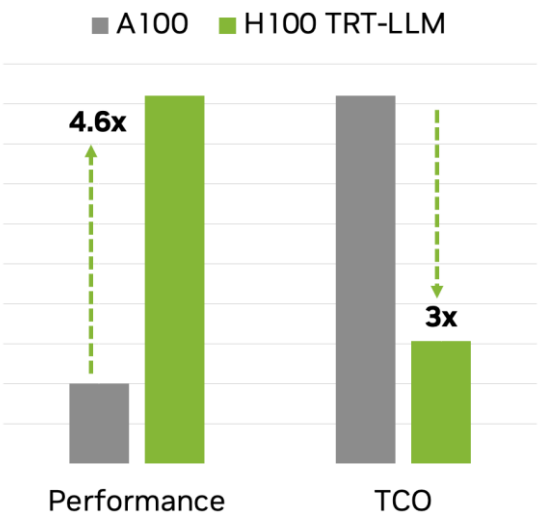


TensorRT-LLM Optimizing LLM Inference

SoTA Performance for Large Language Models for Production Deployments

SoTA Performance

Leverage TensorRT compilation & kernels from FasterTransformers, CUTLASS, OAI Triton, ++



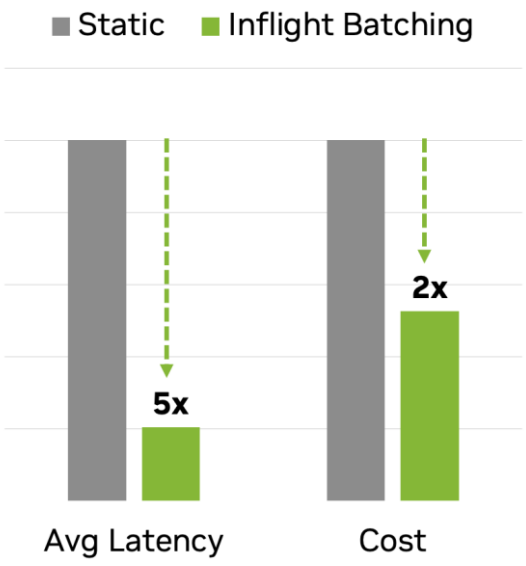
Ease Extension

Add new operators or models in Python to quickly support new LLMs with optimized performance

```
# define a new activation
def silu(input: Tensor) → Tensor:
    return input * sigmoid(input)

#implement models like in DL FWs
class LlamaModel(Module)
  def __init__(…)
    self.layers = ModuleList([…])

  def forward (…)
    hidden = self.embedding(…)

    for layer in self.layers:
      hidden_states = layer(hidden)

    return hidden
```

LLM Batching with Triton

Maximize throughput and GPU utilization through new scheduling techniques for LLMs



Numbers are preliminary based on internal evaluation on Llama 7B on H100

Triton Inference Server

Open-Source Software For Fast, Scalable, Simplified Inference Serving

Any Framework

Supports Multiple Framework Backends Natively e.g., TensorFlow, PyTorch, TensorRT, XGBoost, ONNX, Python & More

Any Query Type

Optimized for Real Time, Batch, Streaming, Ensemble Inferencing

Any Platform

X86 CPU | Arm CPU | NVIDIA GPUs | MIG

Linux | Windows | Virtualization

Public Cloud, Data Center and Edge/Embedded (Jetson)

DevOps & MLOps

Integration With Kubernetes, KServe, Prometheus & Grafana

Available Across All Major Cloud AI Platforms

Performance & Utilization

Model Analyzer for Optimal Configuration

Optimized for High GPU/CPU Utilization, High Throughput & Low Latency

Stress-testing a client-server system

Simulate many client LLM requests

1. Simulate LLM requests

2. Measure time to get response
to calculate latency and throughput

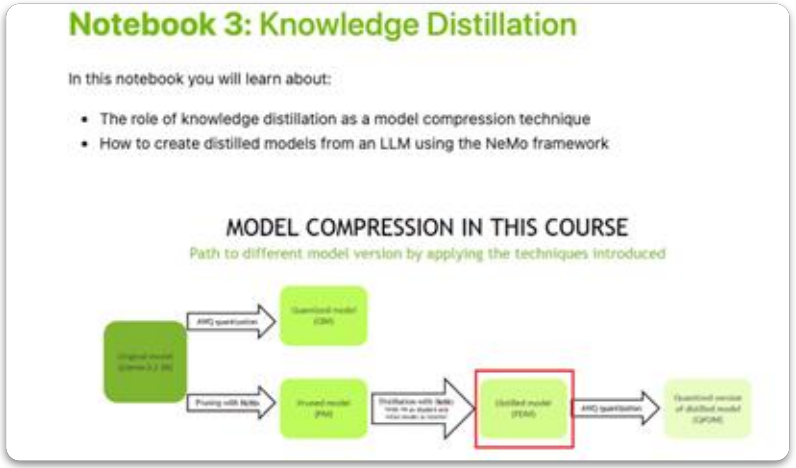
Client

Server

The Lab

Notebooks 3 + 4

- **Notebook 3:**
  - Run the knowledge distillation and open tensorboard
  - Evaluate the distilled model on MMLU Benchmark



- **Notebook 4:**
  - Build TensorRT-LLM engines of the distilled engine
  - Launch the Triton server with the engine
  - Measure the speed of the engine

NVIDIA